Universidad del Valle de Guatemala
Facultad de Ingeniería
Teoría de la Computación

Ciudad de Guatemala, Guatemala
17 de octubre de 2026

Algoritmo CYK

Proyecto 2



DELVALLE
GRUPO EDUCATIVO

Juan Francisco Orozco Mijangos (24647)
Antony David Melquisedec Saz Saz (24710)

1. Descripción del proyecto

En este proyecto implementamos un programa en Python que determina si una frase sencilla en inglés pertenece al lenguaje generado por una gramática libre de contexto. Para realizar la validación utilizamos el algoritmo CYK, que trabaja mediante programación dinámica y requiere una gramática en Forma Normal de Chomsky.

El programa realiza la simplificación y conversión de la gramática antes de analizar la frase. Como resultado, muestra SÍ o NO, el tiempo utilizado durante la validación y un árbol sintáctico cuando la entrada es aceptada.

La validación se limita a la estructura definida por la gramática. Por lo tanto, una frase puede ser aceptada aunque su significado resulte extraño.

2. Diseño de la aplicación

Organizamos el programa en cuatro componentes principales:

Archivo	Responsabilidad
main.py	Recibe la entrada, coordina las etapas y presenta los resultados.
cyk/grammar.py	Lee y representa la gramática, sus símbolos y producciones.
cyk/cnf.py	Simplifica la gramática y la convierte a Forma Normal de Chomsky.
cyk/parser.py	Ejecuta CYK, mide el tiempo y construye el árbol sintáctico.

El recorrido general de la aplicación es:

Archivo de gramática → Lectura → Simplificación → Conversión a CNF → CYK → Resultado, tiempo y árbol.

La frase ingresada se separa en palabras y símbolos antes de enviarse al algoritmo.

2.1 Representación de la gramática

Utilizamos la clase Grammar para almacenar el símbolo inicial y las producciones. Cada producción se representa como una tupla de símbolos, mientras que la tupla vacía representa epsilon.

Los no terminales se identifican por su aparición en el lado izquierdo de las producciones. Los demás símbolos se consideran terminales. Esta decisión permite trabajar con nombres como NP, VP y Det sin depender de que todos estén escritos completamente en mayúsculas.

Los archivos admiten alternativas separadas por una barra vertical y comentarios que empiezan con el símbolo numeral.

2.2 Simplificación de gramáticas

Implementamos tres etapas de simplificación.

Eliminación de producciones epsilon

Primero identificamos los símbolos que pueden generar la cadena vacía. Repetimos la búsqueda hasta que no aparezcan nuevos símbolos anulables.

Después generamos las variantes de cada producción que resultan de omitir esos símbolos. Si una regla contiene varias posiciones anulables, consideramos todas sus combinaciones. La producción epsilon se conserva para el símbolo inicial cuando el lenguaje original incluye la cadena vacía.

Eliminación de producciones unitarias

Las producciones unitarias tienen la forma $A \rightarrow B$, donde ambos símbolos son no terminales. Para eliminarlas, seguimos las cadenas de reglas unitarias y copiamos las producciones no unitarias que pueden alcanzarse desde cada símbolo.

También controlamos los símbolos visitados para evitar recorridos indefinidos cuando existen ciclos.

Eliminación de símbolos inútiles

Primero identificamos los símbolos capaces de generar cadenas de terminales. Luego buscamos cuáles son alcanzables desde el símbolo inicial. Conservamos únicamente los que cumplen ambas condiciones.

Aplicamos las etapas en ese orden porque eliminar epsilon puede producir nuevas reglas unitarias.

2.3 Conversión a Forma Normal de Chomsky

Después de simplificar, transformamos las producciones para que tengan alguna de estas formas:

- $A \rightarrow B\ C$, con dos no terminales.
- $A \rightarrow a$, con un terminal.
- $S \rightarrow \varepsilon$, únicamente cuando corresponde conservar la cadena vacía y el inicio no aparece a la derecha.

Cuando un terminal aparece dentro de una producción de varios símbolos, lo reemplazamos por un no terminal auxiliar. Por ejemplo, $A \rightarrow a\ B$ se transforma en $A \rightarrow T\ B$ y $T \rightarrow a$.

Las producciones con más de dos símbolos se dividen en reglas binarias. Por ejemplo, $A \rightarrow B\ C\ D$ se transforma en $A \rightarrow B\ X$ y $X \rightarrow C\ D$.

Los nombres auxiliares se generan sin repetir símbolos existentes. Al terminar, verificamos que la gramática resultante cumpla las restricciones de CNF.

La gramática inglesa del enunciado ya contiene producciones binarias o terminales. Por eso también utilizamos la gramática aritmética proporcionada para comprobar una conversión que sí necesita simplificación y transformación.

2.4 Algoritmo CYK

CYK construye una tabla donde cada celda contiene los no terminales capaces de generar un fragmento de la entrada.

Primero llenamos la diagonal con los no terminales que producen cada palabra. Después analizamos fragmentos de longitud creciente. Para cada fragmento probamos todos sus posibles puntos de división y combinamos los resultados de sus dos partes.

Si encontramos una regla A → B C y las dos partes pueden ser generadas por B y C, agregamos A a la celda correspondiente.

La frase se acepta cuando el símbolo inicial aparece en la celda que representa toda la entrada.

Esta organización utiliza programación dinámica porque los resultados de los fragmentos pequeños se calculan una vez y se reutilizan al construir fragmentos mayores. Con una gramática fija, el tiempo de ejecución es $O(n^3)$ y el espacio es $O(n^2)$, donde n es la cantidad de tokens.

2.5 Árbol sintáctico y medición del tiempo

Cada celda guarda nodos con las derivaciones encontradas. Los nodos de la diagonal contienen terminales y los nodos que combinan fragmentos conservan referencias a sus dos hijos.

Cuando la frase es aceptada, obtenemos el nodo del símbolo inicial y recorremos sus hijos para mostrar el árbol. Si existen varias derivaciones, conservamos una de ellas siguiendo un orden estable.

Para una gramática transformada se ocultan los auxiliares introducidos durante la conversión. No se reconstruyen los pasos unitarios o epsilon eliminados. En la gramática inglesa del enunciado esto no modifica su estructura, porque ya está en CNF.

El tiempo se mide con perf_counter_ns e incluye la construcción y el llenado de la tabla CYK. La lectura de archivos, conversión de la gramática, tokenización e impresión quedan fuera de esa medición. El resultado se presenta en nanosegundos y milisegundos.

3. Ejemplos y pruebas realizadas

3.1 Frases aceptadas

Frase	Resultado	Explicación
She eats a cake with a fork.	SÍ	Contiene un sujeto, una frase verbal y una frase preposicional.

The cat drinks the beer.	SÍ	Sigue la estructura de sujeto y verbo con objeto.
He cooks.	SÍ	La gramática permite un pronombre seguido de un verbo sin objeto.
A dog eats the soup in the oven.	SÍ	Sus componentes pueden derivarse con las reglas proporcionadas.

Para “The cat drinks the beer”, el árbol tiene como raíz S, que se divide en NP y VP. El primer NP contiene “the cat”. El VP contiene el verbo “drinks” y otro NP correspondiente a “the beer”.

La última frase de la tabla muestra que la aceptación depende de las reglas sintácticas, no de que el significado sea razonable.

3.2 Frases rechazadas

Frase	Resultado	Explicación
Cake she eats a.	NO	El orden no puede generarse con las producciones disponibles.
She likes cake.	NO	La palabra “likes” no pertenece al vocabulario.
The cat the beer drinks.	NO	La ubicación del verbo no corresponde a la estructura de la gramática.
She eats with.	NO	La preposición necesita una frase nominal después.

Cuando una frase es rechazada, el programa muestra NO e indica que no existe un árbol sintáctico completo para esa entrada.

3.3 Pruebas con la gramática aritmética

Utilizamos las reglas de 1.txt y la versión de referencia de 1-cnf.txt para comprobar la conversión.

Expresión	Resultado
id + id * id	SÍ
(id+id)*id	SÍ
id + * id	NO

La CNF generada puede tener nombres auxiliares y cantidades de producciones diferentes de la referencia. Para comprobar la conversión verificamos que ambas reconozcan las mismas entradas dentro del conjunto evaluado.

3.4 Pruebas automatizadas

La aplicación cuenta con 30 pruebas automatizadas. Para ejecutarlas utilizamos:

python3 -m unittest discover -s tests -v

El resultado obtenido fue:

Ran 30 tests — OK

Las pruebas incluyen reconocimiento de frases, reconstrucción de árboles, eliminación de epsilon, reglas unitarias, símbolos inútiles y validación de CNF. También cubren ciclos, símbolos anulables repetidos, lenguaje vacío, nombres auxiliares ocupados y errores de archivos.

Para comprobar la conservación del lenguaje implementamos una derivación por punto fijo independiente de CYK. Con ella generamos las cadenas de diez gramáticas de prueba hasta cuatro tokens y comparamos los resultados antes y después de convertirlas.

Para la gramática aritmética evaluamos las 3,906 secuencias posibles de hasta cinco tokens sobre el alfabeto formado por id, +, *, (y), incluida la cadena vacía. Comparamos el reconocimiento de la CNF generada y de la referencia con las cadenas derivadas directamente de la gramática original. Dentro de ese límite se aceptaron 15 entradas y se rechazaron 3,891.

Además, verificamos que las hojas de cada árbol obtenido coincidan con los tokens de la entrada. Estas comparaciones son exhaustivas dentro de las longitudes indicadas.

4. Discusión

4.1 Dificultades y decisiones de implementación

Una dificultad fue manejar correctamente epsilon cuando varios símbolos de una misma producción podían desaparecer. Para conservar las posibilidades de derivación fue necesario generar todas las combinaciones de omisiones.

También debimos distinguir el marcador “e” del no terminal “E” de la gramática aritmética. El lector conserva esa diferencia para evitar interpretar una expresión como cadena vacía.

La construcción del árbol requirió guardar más información que una respuesta booleana. Al conservar nodos con referencias a sus hijos, pudimos recuperar una derivación completa después de llenar la tabla.

Otra decisión fue mantener los símbolos desconocidos de la entrada. Por ejemplo, “she_eats a cake” debe rechazarse, en lugar de convertirse silenciosamente en “she eats a cake”. En cambio, normalizamos las mayúsculas y permitimos puntuación final en las frases inglesas.

Finalmente, controlamos los nombres de los símbolos auxiliares para evitar colisiones con los de la gramática original.

4.2 Recomendaciones

Recomendamos separar los símbolos con espacios en los archivos de gramática y ejecutar las pruebas después de modificar las producciones.

Para explicar el funcionamiento durante una demostración, conviene utilizar las opciones --show-cnf y --show-table. La primera permite revisar la gramática transformada y la segunda muestra cómo se resolvieron los fragmentos de la frase.

Al comparar tiempos es necesario repetir las ejecuciones, ya que las entradas pequeñas producen mediciones cortas que pueden variar según la carga del equipo.

También es importante distinguir entre lenguaje vacío y cadena vacía. Una gramática que genera el lenguaje vacío no acepta ninguna entrada; una que genera epsilon sí acepta una entrada sin tokens.

5. Conclusiones

Implementamos las etapas necesarias para simplificar una gramática, convertirla a Forma Normal de Chomsky y aplicar CYK mediante programación dinámica.

El programa permite comprobar la pertenencia de una frase, medir el tiempo de validación y visualizar una derivación mediante un árbol sintáctico. Las pruebas con inglés y expresiones aritméticas permitieron revisar tanto el reconocimiento como la transformación de las gramáticas.

El proyecto nos permitió relacionar las reglas formales de una gramática con un procedimiento que construye resultados a partir de subproblemas y conserva la información necesaria para explicar una aceptación.

6. Referencias

- University of Nevada, Las Vegas. *The CYK Algorithm*. Apuntes de Automata and Formal Languages. https://web.cs.unlv.edu/larmore/Courses/CSC456/cyk.html

Repositorio en Github: https://github.com/antonySAZ7/Proyecto2-teo-compu